Capstone 1 Report: Urban Tree Cover Analysis

Report 1: Case Study Selection

Research Question

How do European NUTS2 regions compare in urban tree cover relative to their population?

Scope

- **Regions:** 89 NUTS2 regions with population and tree cover data
- **Data Source:** Eurostat population + Urban Atlas 2021 boundaries + Copernicus tree raster
- **Geographic Coverage:** EU/EFTA metropolitan areas

Why This Question

- Links population density to green infrastructure
- Identifies regions with high/low tree cover equity
- Informs urban planning policy

Deliverable

Rankings of regions by tree cover percentage and km²

Report 2: Data Source Identification

Data Sources

Source	File	Records	Description
Population	estat_tgs00096.tsv.gz	227	Eurostat NUTS population
NUTS Mapping	NUTS2021.xlsx	229	NUTS2 to metro label
FUA Boundaries	Results/2021/*.zip	768	Urban Atlas 2021
Tree Raster	TCD_2015_020m_eu_03035_d05_Full.tif	1	Copernicus TCD 2015

Population Data

- **Source:** Eurostat TGS00096
- **Format:** TSV (tab-separated)
- **Years:** 2014-2025 (latest: 2024)
- **Geographic Units:** NUTS2 codes (e.g., AT13, DE11)

NUTS Mapping

- **Source:** NUTS2021.xlsx - Metropolitan sheet
- **Purpose:** Maps NUTS codes to city/region names
- **Example:** AT13 → Wien (Vienna)

FUA Boundaries

- **Source:** Copernicus Urban Atlas 2021
- **Format:** GeoPackage (.fgb) in ZIP files
- **Coverage:** 768 metropolitan areas

Tree Raster

- **Source:** Copernicus Tree Cover Density 2015
- **Resolution:** 20m
- **Projection:** EPSG:3035 (ETRS89/LAEA)

Report 3: Data Collection Strategy

Pipeline Overview

Two-stage pipeline:

1. **Stage 1 (Import/Clean):** Read sources → merge → save to DB
2. **Stage 2 (Analysis):** Read DB → filter → output rankings

Stage 1: Import/Clean

```
INPUTS:
- estat_tgs00096.tsv.gz (population)
- NUTS2021.xlsx (mapping)
- Results/2021/*.zip (FUA boundaries)
- TCD_2015...tif (tree raster)

PROCESS:
1. Parse TSV → extract NUTS code + population
2. Parse NUTS2021.xlsx → NUTS2 → city name mapping
3. Match regions → merge population + FUA
4. Extract tree cover using FUA geometry
5. Save to SQLite DB

OUTPUT:
- regions table with: name, nuts_code, population, area_km2, tree_cover_km2,
tree_cover_pct
```

Stage 2: Analysis

```
INPUT: SQLite DB

PROCESS:
1. Query regions from DB
2. Filter by population threshold
3. Filter by major cities list

OUTPUT: Rankings by tree cover km2 and percentage
```

Key Processing Steps

1. **Name Normalization:** Wien → vienna, Bruxelles → brussels, etc.
2. **FUA Matching:** Via normalized name matching (89 matched)
3. **Tree Extraction:** Clip raster to actual FUA geometry (not fixed buffer)

Report 4: Data Collection (Implementation)

Implementation

import_data.py

```
BASE_DIR = os.environ.get("KEMV_DATA_DIR", os.path.dirname(os.path.abspath(__file__)))

# 1. Read population TSV
with gzip.open(POP_PATH, 'rt') as f:
    for line in f:
        parts = line.strip().split('\t')
        geo = parts[0].split(',')[-1]
        if re.match(r'^[A-Z]{2}[0-9]{2,3}$', geo):
            for val in reversed(parts[1:]):
                try:
                    pop_data[geo] = int(val.strip())
                    break
                except: continue

# 2. Read NUTS mapping from Excel
df = pd.read_excel(NUTS_XLSX, sheet_name='Metropolitan')
metro = df[df['METRO (No/Yes)'] == 'Y']
nuts_map = {}
for _, row in metro.iterrows():
    nuts_map[str(row['NUTS ID'])[:4]] = row['METRO LABEL']

# 3. Merge population with NUTS mapping
for nuts, pop in pop_data.items():
    name = nuts_map.get(nuts[:4], nuts)
    key = norm(name)
    merged[key] = (nuts, name, pop)

# 4. Match FUA and extract tree cover
with tempfile.TemporaryDirectory() as tmpd:
    with zipfile.ZipFile(zf_path, 'r') as z:
        z.extractall(tmpd)
    with fiona.open(fgb, 'r') as src:
        geoms = [shape(f['geometry']) for f in src if shape(f['geometry']).is_valid]
[:500]
    union = unary_union(geoms)
    area_km2 = union.area / 1_000_000

    with rasterio.open(RASTER_PATH) as rst:
        bbox = union.bounds
        window = rasterio.windows.from_bounds(bbox, rst.transform)
        col_off = int(round(window.col_off))
        row_off = int(round(window.row_off))
        width = int(round(window.width))
        height = int(round(window.height))
        window = rasterio.windows.Window(col_off, row_off, width, height)
        data = rst.read(window=window)
        mask = geometry_mask([union], (height, width),
            rst.window_transform(window), invert=True)
```

```
        tree_pix = int(np.sum((data[0] > 0) & mask))
        tree_km2 = tree_pix * PIXEL_KM2

# 5. Save to DB
c.execute("INSERT INTO regions ...", (name, nuts, pop, area_km2, tree_km2, tree_pct))
```

Output

- 212 regions in database
- 89 with tree cover data extracted from actual FUA boundaries

Performance

- Processed 89 cities in ~10 minutes
- Limited to 500 features per FUA for speed

Report 5: Data Cleaning

Cleaning Operations

1. Name Normalization

Applied to match NUTS codes with FUA boundaries:

Original	Normalized
Wien	vienna
Praha	prague
Lefkosia	nicosia
Bruxelles	brussels
Bucuresti	bucharest
Grad Zagreb	zagreb

2. NUTS Mapping

- Source: NUTS2021.xlsx Metropolitan sheet
- Logic: Map NUTS2 (first 4 chars) to city name
- Example: AT13 → Wien

3. FUA Matching

- Method: Normalized name matching
- Matched: 89 out of 212 regions
- Unmatched: Regions without FUA boundary data

4. Tree Extraction

- Use actual FUA geometry from Urban Atlas 2021
- Clip tree raster to FUA boundary
- Calculate: tree_km2 / area_km2 * 100 = percentage
- Limited to 500 features per FUA for performance

Results

Metric	Count
Total regions	212
With population	212
With FUA match	89
With tree data	89

Sample Results

Rank	Region	Pop	Area km²	Tree km²	Tree %
1	Ostrava	1.2M	95	88	92.6%
2	Brasov	2.3M	316	275	86.9%
3	Karlsruhe	2.8M	63	52	82.8%